SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

SSAFY

SW문제해결 응용

시작하기

Python

학습시작

아래 문장은 컴퓨터 입장에서 본 SW 문제해결을 시작하는 방법입니다.

1. 문제를 해결할 때 항상 [ ] 절차를 따릅니다.
2. 사람들은 나를 빠르게 만들기 위해 [ ] 를 분석해서 가능한 적은 [ ] 안에 계산이 끝나도록 알고리즘을 설계해줍니다.
3. 어떤 입력이 주어져도 [ ] 에 따라 정확히 실행만 하면 됩니다.

- SW 문제해결 능력을 키우면 효율적인 알고리즘 설계, 복잡도 분석, 비트·진수·입출력 등 컴퓨터적 사고의 기초를 다질 수 있습니다.
- 향후 다양한 문제 상황에서도 정확하고 빠르게 프로그램을 구현할 수 있는 기반이 됩니다.

Objectives

학습목표

- SW 문제해결 역량과 강화 방법을 이해한다.
- 효율적인 알고리즘의 필요성과 시간복잡도에 대해 이해한다.
- 비트 수준의 연산과 자주 활용되는 표현에 대해 학습한다.
- 다양한 진법들과 이들 사이에 변환 방법을 이해한다.
- 컴퓨터에서 실수 자료형의 표현과 특징을 이해한다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

목 차

시작하기

- SW 문제해결
- 복잡도 분석
- 표준 입출력 방법

진법과 연산

- 진법
- 진법 변환
- 비트 연산
- 실수
- 연습 문제

시작하기

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

SW 문제해결

SW 문제해결 역량이란?

- 프로그램을 하기 위한 많은 제약 조건과 요구 사항을 이해하고 최선의 방법을 찾아내는 능력
- 프로그래머가 사용하는 언어나 라이브러리, 자료구조, 알고리즘에 대한 지식을 적재적소에 퍼즐을 배치하듯 이들을 연결하여 큰 그림을 만드는 능력이라 할 수 있음
- 문제해결 역량은 추상적인 기술임
  - 프로그래밍 언어, 알고리즘처럼 명확히 정의된 실체가 없습니다.
  - 무작정 알고리즘을 암기하고 문제를 풀어본다고 향상되지 않습니다.
- 문제해결 역량을 향상시키기 위해서 훈련이 필요함

문제해결 과정

1. 문제를 읽고 이해한다.
2. 문제를 익숙한 용어로 재정의한다.
3. 어떻게 해결할지 계획을 세운다.
4. 계획을 검증한다.
5. 프로그램으로 구현한다.
6. 어떻게 풀었는지 돌아보고, 개선할 방법이 있는지 찾아본다.

리스트에 숫자 넣기

- 수 N을 입력받는다.
- 윗줄에는 N부터 1씩 증가되는 숫자 4개를 왼쪽에 채운다.
- 아랫줄에는 N부터 1씩 감소되는 숫자 4개를 오른쪽에 채운다.
- 최종 결과를 출력한다. 빈 공간은 0으로 출력한다.

ex) N = 5 입력 시

5	6	7	8	0	0	0
0	0	0	2	3	4	5

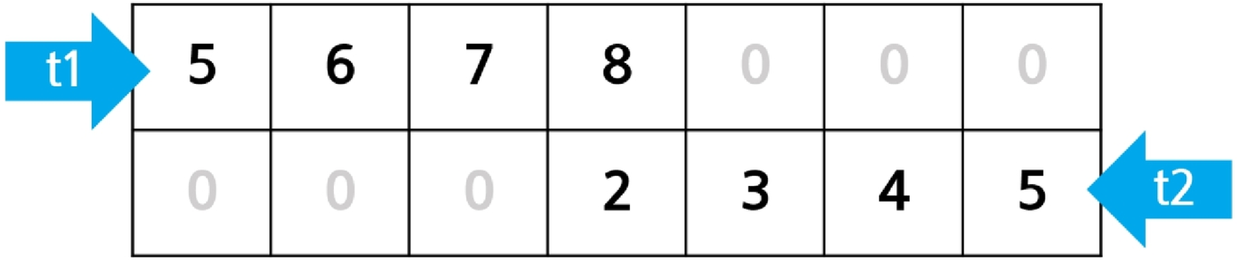
<그림1_SW문제해결 응용_시작하기_리스트에 숫자 넣기>

문제를 이해하고 종이와 펜을 이용하여 직접 설계까지 해보자. (3분)

설계 예시

- 2 x 7 크기의 0으로 채운 리스트를 준비한다.
- 수 N을 입력받는다.
- t1 = N 대입 후, for문을 4번 돌려 t1 값을 리스트에 넣는다. range(0, 4)
- t2 = N 대입 후, for문을 4번 돌려 t2 값을 리스트에 넣는다. range(6, 2, -1)
- 리스트 출력

ex) N = 5 입력 시



<그림2_SW문제해결 응용_시작하기_리스트에 숫자 넣기2>

SW 문제해결 능력을 기르는 이유

- 코딩을 더 잘하게 하는 능력을 기르기 위함
- SWEA 문제를 통해 기본문법 & 자료구조 & 알고리즘을 더 능숙하게 쓸 수 있도록 훈련함

SWEA 문제를 잘 풀기 위한 전략

- 단계 1 : 완벽한 문제 이해
- 단계 2 : 종이와 펜을 이용한 설계하기 (어떻게 구현할지 계획하기)
- 단계 3 : 설계한 대로 구현 & 디버깅을 한다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

복잡도 분석

알고리즘이란?

- 유한한 단계를 통해 **문제를 해결하기 위한 절차나 방법**

  - 주로 컴퓨터 용어로 쓰이며, 컴퓨터가 어떤 일을 수행하기 위한 단계적 방법을 말합니다.

- 다시 말하면 어떠한 문제를 해결하기 위한 절차라고 볼 수 있음
- 1부터 100까지의 합을 구하는 두 가지 알고리즘의 예

```
    1
+   2
+   3
.....
+ 100
-------
5,050
```

$$\frac{100 \times (1+100)}{2} = 5,050$$

<그림3_SW문제해결 응용_시작하기_알고리즘 절차 예시>

알고리즘의 효율

- 공간적 효율성과 시간적 효율성

- 공간적 효율성은 알고리즘이 필요로 하는 메모리 공간을 말합니다.
- 시간적 효율성은 알고리즘이 작업을 완료하는 데 걸리는 시간을 말합니다.
- 효율성을 뒤집어 표현하면 복잡도(Complexity)가 됩니다. 복잡도가 높을수록 효율성은 저하됩니다.

※ 시간적 효율성은 주로 입력 크기 n에 대한 연산 횟수로 나타냅니다.

복잡도의 점근적 표기

- 시간(또는 공간)복잡도는 입력 크기에 대한 함수로 표기
  - 이 함수는 주로 여러 개의 항을 가지는 다항식입니다.
  - 이를 단순한 함수로 표현하기 위해 점근적 표기(Asymptotic Notation)를 사용합니다.

- 입력 크기 n이 무한대로 커질 때의 복잡도를 간단히 표현하기 위해 사용하는 표기법
  - O(Big-Oh 또는 Big-O) 표기
  - Ω(Big-Omega) 표기
  - Θ(Big-Theta) 표기

아래 코드는 몇 번 반복될까?

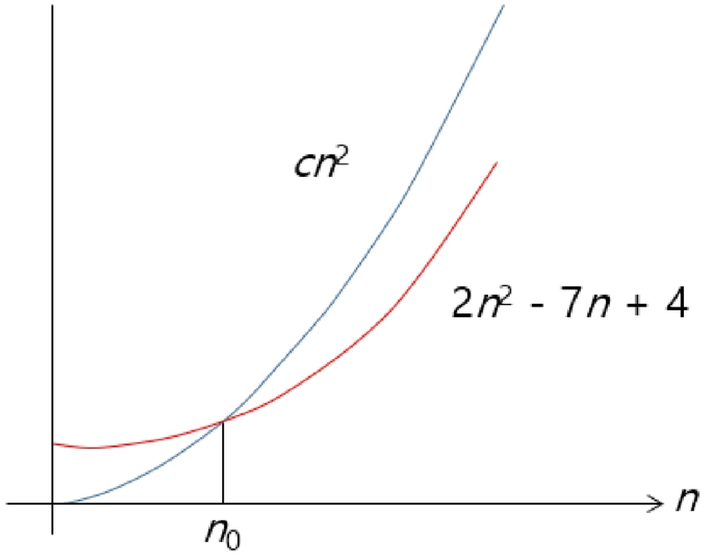
```
n = int(input())

for i in range(n):
    print(i, end = '')
```

<그림4_SW문제해결 응용_시작하기_복잡도 분석>

O(Big-Oh) 표기

- Big-Oh 표기는 복잡도의 점근적 상한을 나타냄
- 복잡도가 $f(n) = 2n^2-7n+4$ 이라면, f(n)의 O-표기는 $O(n^2)$
- 먼저 f(n)의 단순화된 표현은 n^2
- 단순화된 함수 n^2에 임의의 상수 c를 곱한 cn^2이 n이 증가함에 따라 f(n)의 상한이 된다. (단, c>0)



<그림5_SW문제해결 응용_시작하기_O(Big-O)표기>

빅오표기법으로 표현하면?

- O(n)

main.py

```
1   n = int(input())
2
3   for i in range(n):
4       print(i, end = '')
5
6   for i in range(n):
7       print(i, end = '')
8
9   for i in range(n):
10      print(i, end = '')
11
```

<그림6_SW문제해결 응용_시작하기_O(Big-O)표기2>

빅오표기법으로 표현하면?

- $O(n^2)$

```
n = int(input())

for i in range(n):
    for x in range(n):
        print(i, end = '')

    for y in range(n):
        print(i, end = '')
```

<그림7_SW문제해결 응용_시작하기_O(Big-O)표기3>

상수 횟수만큼 반복하는 경우 빅오표기법

- O(1)

```
n = int(input())

for i in range(50):
    print(i)
```

<그림8_SW문제해결 응용_시작하기_O(Big-O)표기4>

빅오표기법 표현 방법

- O(5N)

- 정확한 배수를 강조하고 싶은 경우라면?
- O(N)대신 O(5N)으로 표현할 수 있습니다.

예시 : 알고리즘 성능을 미세하게 비교하고 싶은 경우

기존 알고리즘 성능이 O(7N) 이지만,
제가 짠 알고리즘 성능은 O(2N)으로
더 좋은 성능을 냅니다.

빅오표기법으로 성능 비교하기

- 어떤 알고리즘이 더 좋은 성능을 내는가 ?

O(N) 으로
작성한 알고리즘

VS

$O(N^2)$ 으로
작성한 알고리즘

자주 사용하는 O-표기

O(1)	상수 시간 (Constant time)
O(logn)	로그(대수) 시간 (Logarithmic time)
O(n)	선형 시간 (Linear time)
O(nlogn)	로그 선형 시간 (Log-linear time)
$O(n^2)$	제곱 시간 (Quadratic time)
$O(n^3)$	세제곱 시간 (Cubic time)

O(N) 이해하기

- 만약 N이 10,000 이고 O(N)으로 짠 알고리즘이 있다면, 몇 번 반복하는 프로그램이라고 추정해도 될까?

- 정답 : 10,000회

만약 $O(N^2)$ 라면?

- 만약 N이 10,000 이고 $O(N^2)$으로 짠 알고리즘이 있다면, 몇 번 반복하는 프로그램이라고 추정해도 될까?

- 정답 : 100,000,000회

만약 O(log N) 이라면?

- 만약 N이 10,000이고 O(log N)으로 짠 알고리즘이 있다면,
  몇 번 반복하는 프로그램이라고 추정해도 될까?

  - 약 13.3번

- 컴퓨터 분야에서 log의 밑 수는 10이 아니라 2

  : $2^{13} < 10000 < 2^{14}$

계산기를 이용해서 직접 확인해보세요.

만약 O(log N) 이라면?

- 만약 N이 100,000,000 이고 O(log N)으로 짠 알고리즘이 있다면,
  몇 번 반복하는 프로그램이라고 추정해도 될까?

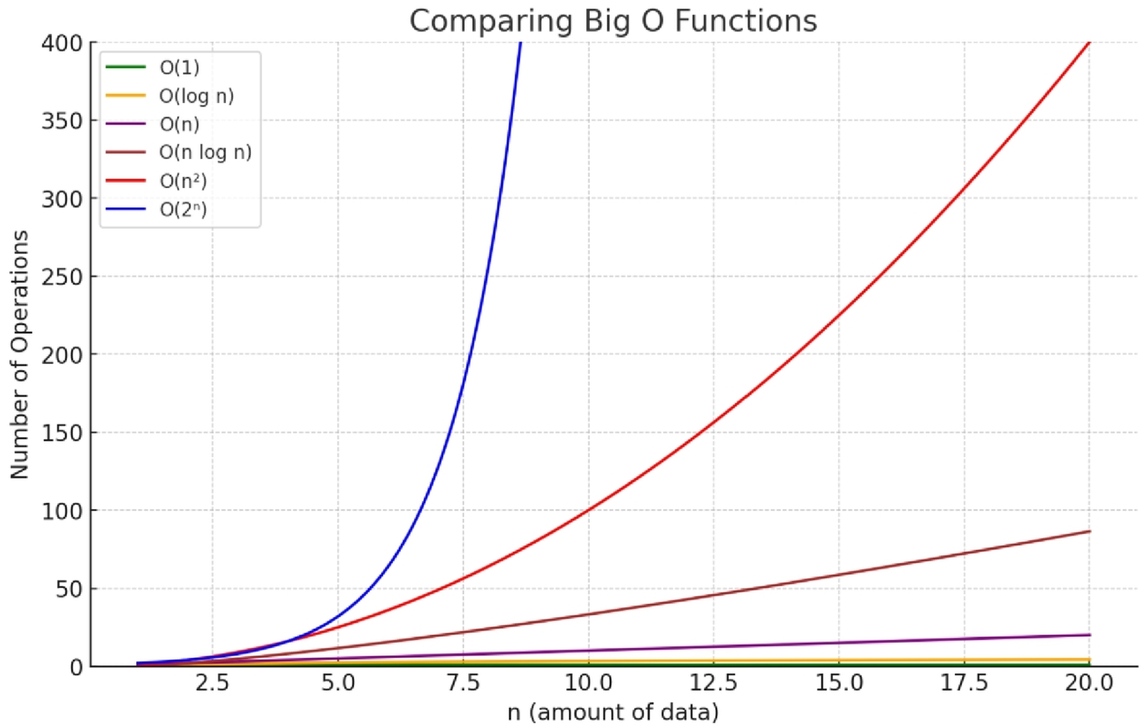
- 2의 몇 승이 1억이 되는지 계산기를 사용해서 알 수 있음

: $2^{26} < 100,000,000 < 2^{27}$

계산기를 이용해서 직접 확인해보세요.

복잡도 비교

- O(logN) 은 O(1) 보다는 느리지만, N이 작은 경우 유사한 성능
- O(NlogN) 은 O(N) 보다는 느리지만, N이 작은 경우 유사한 성능



<그림9_SW문제해결 응용_시작하기_O(Big-O)표기5>

왜 효율적인 알고리즘이 필요한가?

- 10억 개의 숫자를 정렬하는데 PC에서 $O(n^2)$ 알고리즘은 300여 년이 걸리는 반면에 O(nlogn) 알고리즘은 5분 만에 정렬함

$O(n^2)$	1,000	1백만	10억
PC	< 1초	2시간	300년
슈퍼컴	< 1초	1초	1주일
O(nlogn)	**1,000**	**1백만**	**10억**
PC	< 1초	<1초	5분
슈퍼컴	< 1초	<1초	<1초

<표1_SW문제해결 응용_시작하기_효율적인 알고리즘이 필요한 이유>

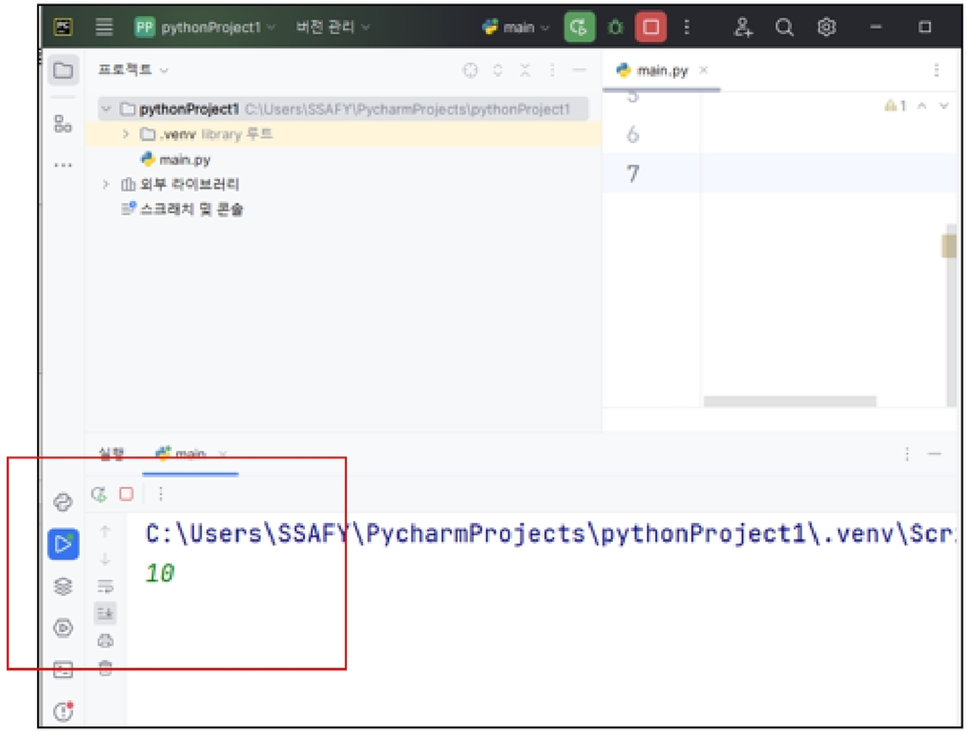
- 효율적인 알고리즘은 슈퍼컴퓨터보다 더 큰 가치가 있음
- 값비싼 HW의 기술 개발보다 효율적인 알고리즘 개발이 훨씬 더 경제적임

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

표준 입출력 방법

챕터의 목표 : 콘솔 입력 대신, 파일 입력으로 입력을 받기



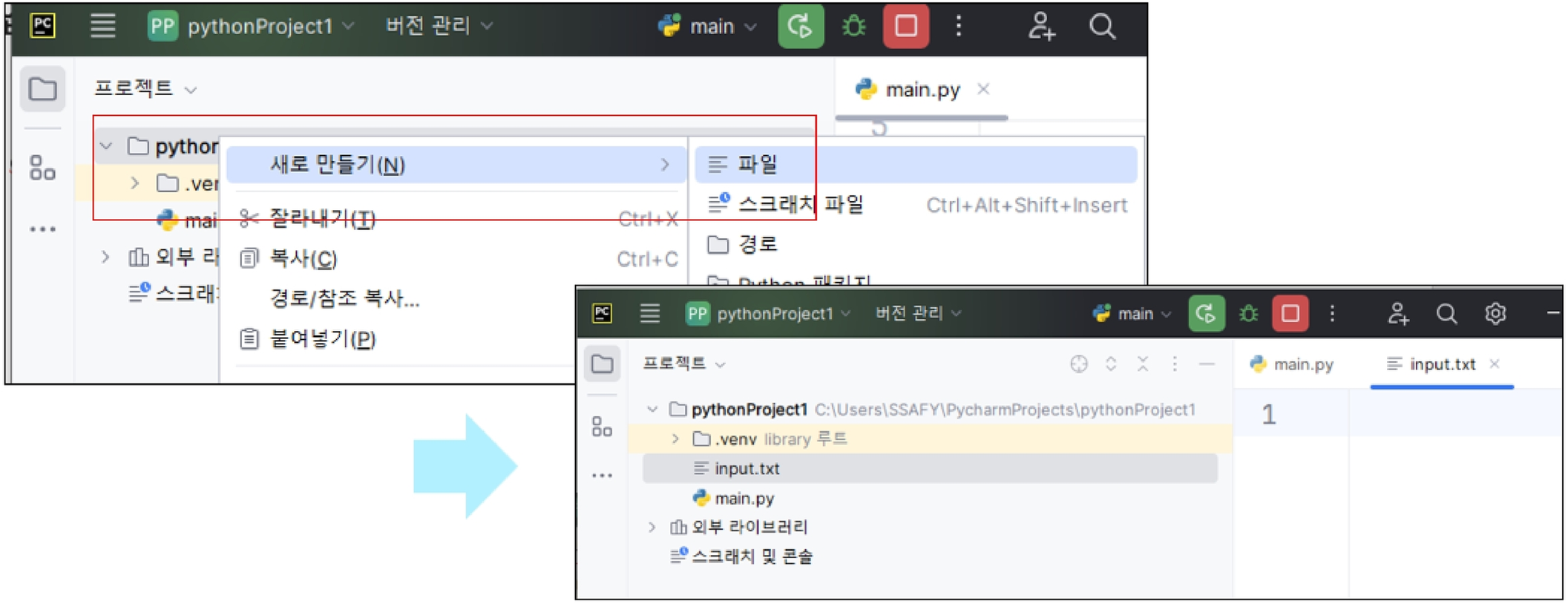
콘솔 입력
(그 동안 써왔던 입력 방법)

파일 입력 방법

<그림10_SW문제해결 응용_시작하기_표준 입출력1>

실습 준비

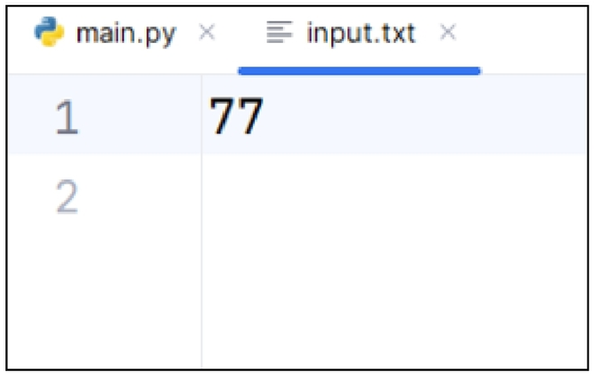
- 하나의 파일을 만들기 : input.txt



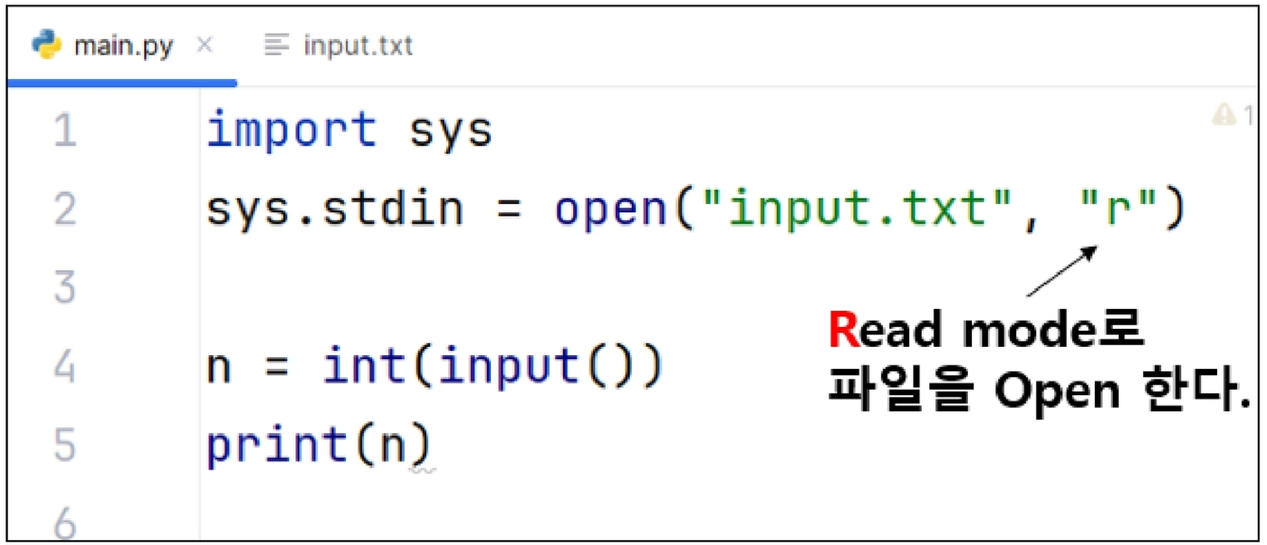
<그림11_SW문제해결 응용_시작하기_표준 입출력2>

실습 준비

- input.txt 에는 77을 적고 저장함
- main.py 에는 우측과 같이 코드를 작성함



input.txt

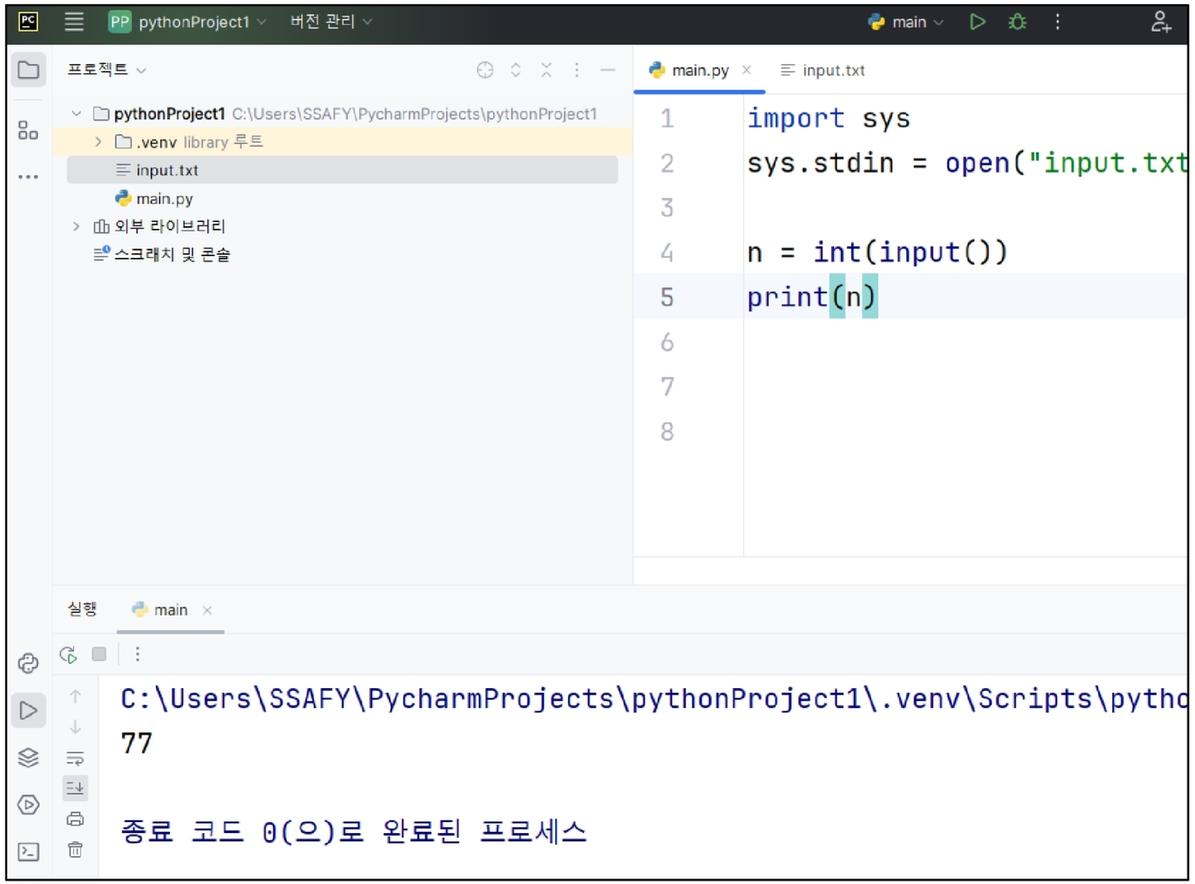


main.py

<그림12_SW문제해결 응용_시작하기_표준 입출력3>

실행 결과

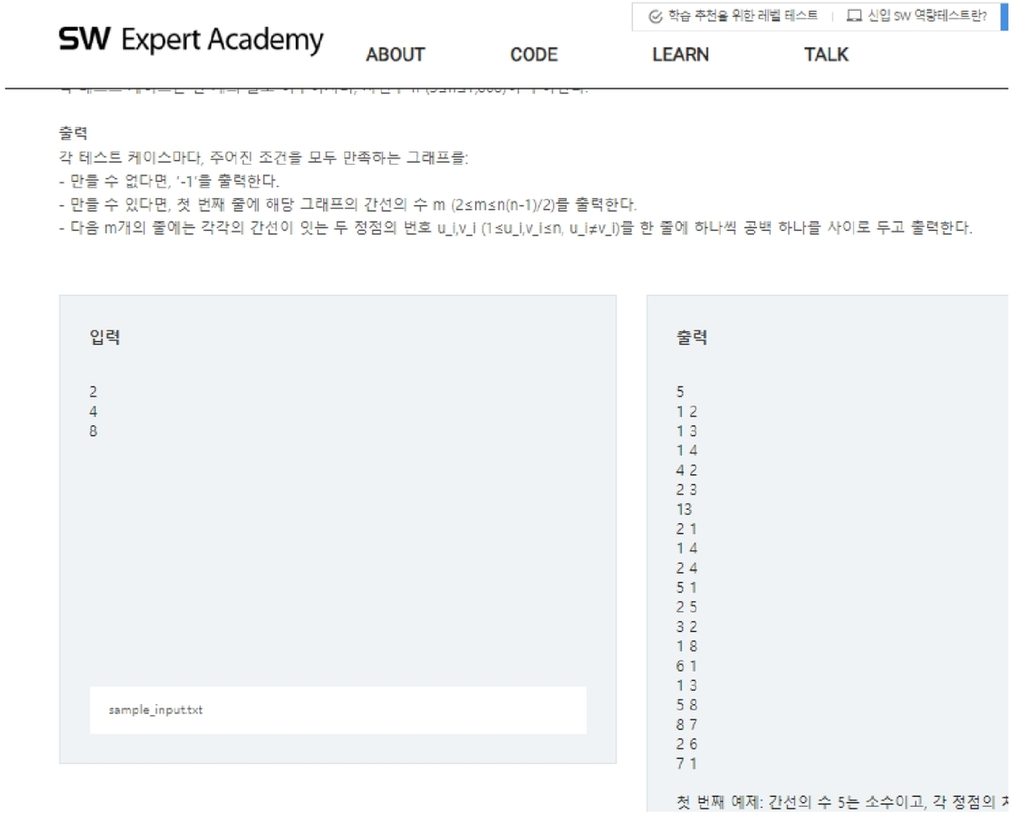
- input()을 하더라도 사용자로부터 입력받지 않고, 텍스트 파일로부터 값을 가져옴



<그림13_SW문제해결 응용_시작하기_표준 입출력4>

SWEA에서 활용하기 1

1. sample_input.txt 내용을 열고 복사하여, input.txt 파일에 붙인다.
2. 문제를 푼다. 테스트 할 때 마다 수동 입력할 필요가 없다.



<그림14_SW문제해결 응용_시작하기_표준 입출력5>

SWEA에서 활용하기 2

3. 코드를 제출할 때는 파일 입력하는 코드를 주석 처리하고 제출한다.

main.py × input.txt

```
1  #import sys
2  #sys.stdin = open("input.txt", "r")
3
4  n = int(input())
5  print(n)
6
7
```

<그림15_SW문제해결 응용_시작하기_표준 입출력6>

SWEA 제출시 주석처리 후에 제출해야 합니다.

[도전] 두 수를 파일에서 입력 받고, 두 수의 합과 곱 출력하기

- 두 수를 input.txt 파일에서 입력받기
- 두 수의 합과 곱을 output.txt 파일에 출력하기

Ex) 2 3 입력 시 출력 결과는 5 6

제한시간 3분

※ 문자열을 입력받고, 문자열을 출력하는 힌트 소스 코드

```
1  import sys
2  sys.stdin = open("input.txt", "r")
3  sys.stdout = open("output.txt", "w")
4
5  text = input()
6  print(text)
```

<그림16_SW문제해결 응용_시작하기_표준 입출력7>

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

진법과 연산

진법

2진수, 8진수, 10진수, 16진수

- 10진수 : 사람이 사용하는 진수, 수 하나를 0~9로 표현
- 2진수 : 컴퓨터가 사용하는 진수, 수 하나를 0, 1로 표현
- 8진수 : 2진수를 더 가독성 있게 사용
- 16진수 : 2진수를 더 가독성 있게 사용, 수 하나를 0, 1, … 8, 9, A, B, C, D, E, F로 표현

$$(168)_{10} = (1010\ 1000)_2$$
$$= (250)_8$$
$$= (A8)_{16}$$

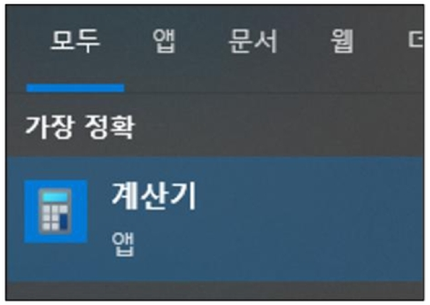
<그림17_SW문제해결 응용_시작하기_진수1>

왜 16진수를 사용하는 것인가?

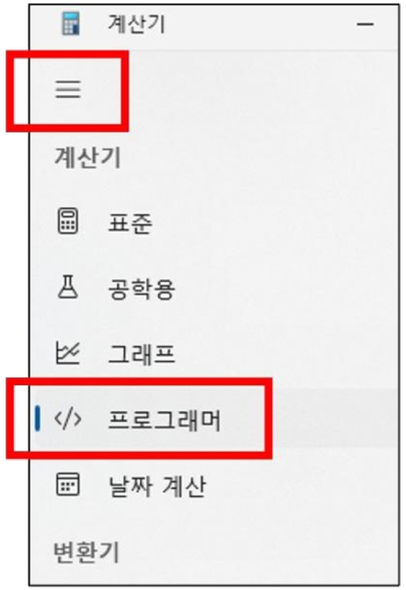
- 2진수를 사람이 이해하기 편하도록, 10진수로 변환 시
  -> 인간이 이해하기 편하지만, 연산이 오래 걸림

- 2진수를 사람이 이해하기 편하도록, 16진수로 변환 시
  -> 인간이 이해하기 어렵지만, 연산 속도가 매우 빠름

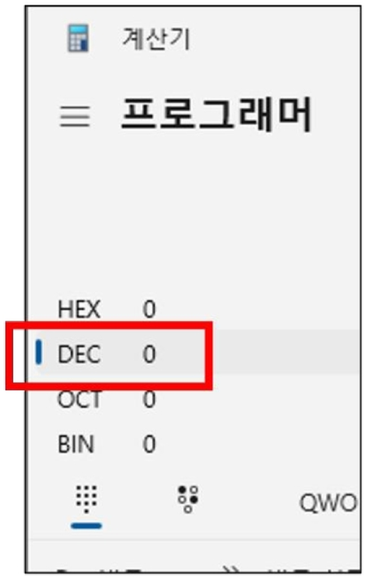
구현하기 전, 검증할 수 있도록 계산기부터 다룰 수 있어야 함



시작 > 계산기



햄버거 버튼 클릭 후
“프로그래머”



DEC 클릭
(DEC : 10진수)

168 입력 후
결과 확인

<그림18_SW문제해결 응용_시작하기_진수2>

용어

- HEX : 16진수
- DEC : 10진수
- OCT : 8진수
- BIN : 2진수

HEX 클릭 후
FF를 눌러보자.

<그림19_SW문제해결 응용_시작하기_진수3>

용어 암기

1. 2진수, 10진수, 16진수를 왜 쓰는지 이해한다. (완료)
2. 손으로 변환하는 방법을 익힌다. (다음 페이지부터 시작)
3. 프로그래밍으로 진법 변환을 직접 구현한다.

실습을 위해 그림판 또는
종이와 펜을 준비해주세요.

SAMSUNG SW AI ACADEMY FOR YOUTH

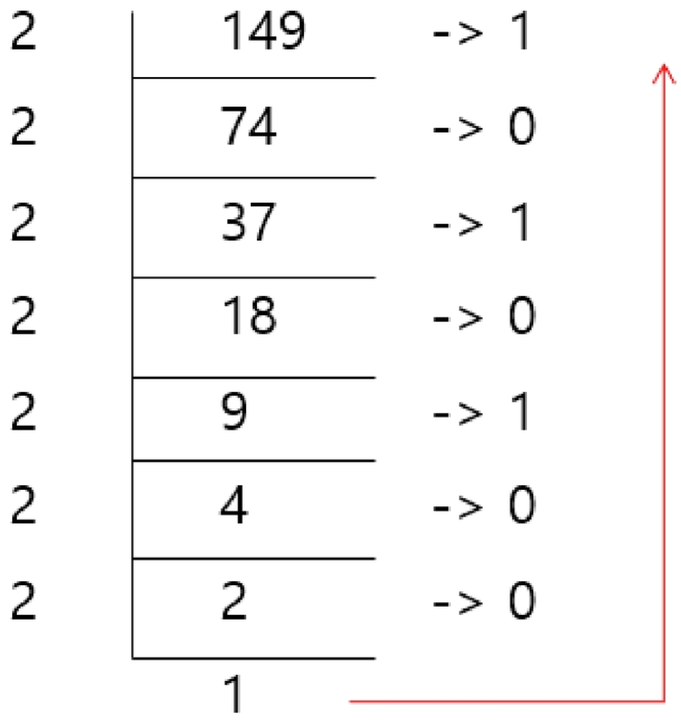
함께가요 미래로!
Enabling People

진법 변환

10진수 -> 다른 진수로 변환

- 원하는 다른 진법의 수로 나눈 뒤 나머지를 거꾸로 읽음

- 예제) $(149)_{10} = (10010101)_2$
  $= (225)_8$
  $= (95)_{16}$



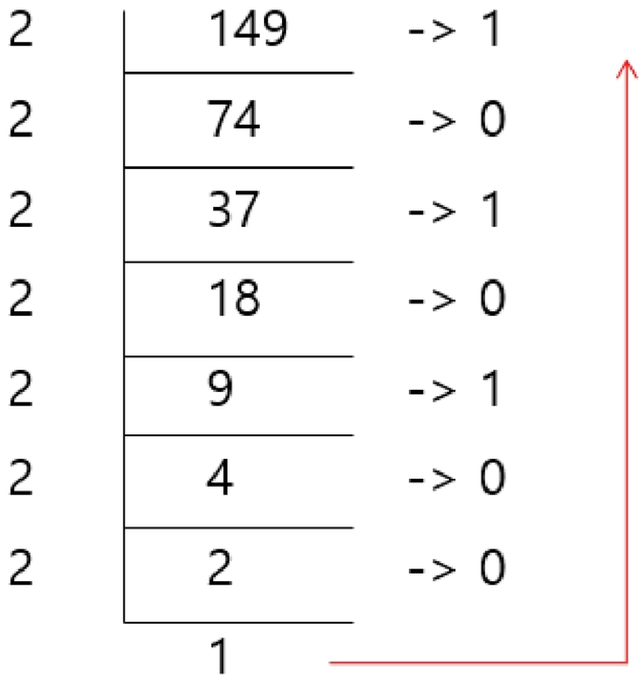
<그림20_SW문제해결 응용_시작하기_진법 변환1>

수기로 10진수를 2진수로 변환해 보기

- 수기로 75를 2진수로 변환하기
- 수기로 14를 2진수로 변환하기

10진수 -> 2 진수로 변환 구현

- 10진수를 지속적으로 2로 나누어 구현
- 마지막으로 List를 거꾸로 뒤집기



<그림20_SW문제해결 응용_시작하기_진법 변환1>

```
tar = 149
result = []

while tar != 0:
    result.append(tar % 2)
    tar //= 2

result.reverse()
print(result)
```

<그림21_SW문제해결 응용_시작하기_진법 변환2>

2진수를 10진수로 변환

- 수기로 계산할 수 있어야 함

$$2^4 \quad 2^3 \quad 2^2 \quad 2^1 \quad 2^0$$

$$1\ 0\ 1\ 1\ 0$$

$$16 \quad 8 \quad 4 \quad 2 \quad 1$$

$$\begin{aligned} & 2^4 * 1 \\ + \ & 2^3 * 0 \\ + \ & 2^2 * 1 \\ + \ & 2^1 * 1 \\ + \ & 2^0 * 0 \end{aligned}$$

$$= 22$$

<그림22_SW문제해결 응용_시작하기_진법 변환3>

16진수와 2진수 변환

- 2진수, 10진수간 변환은 연산이 많으나,
  2진수, 16진수간 변환은 연산이 없음

- 진법 변환을 빠르게 할 수 있으려면
  우측 진수 표를 암기하는 것을 권장함

2진수	10진수	16진수
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	8
1001	9	9
1010	10	a
1011	11	b
1100	12	c
1101	13	d
1110	14	e
1111	15	f

<그림23_SW문제해결 응용_시작하기_진법 변환4>

16진수 0xF9를 2진수로 변환해보기

- 정답 = 1111 1001

16진수 임을 표현하는 두 가지 방법

$(F9)_{16}$

0xF9

숫자 0과 소문자 x

2진수	10진수	16진수
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	8
1001	9	9
1010	10	a
1011	11	b
1100	12	c
1101	13	d
1110	14	e
1111	15	f

<그림23_SW문제해결 응용_시작하기_진법 변환4>

16진수 0xBB3를 2진수로 변환해보기

- 정답 = 1011 1011 0011

2진수	10진수	16진수
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	8
1001	9	9
1010	10	a
1011	11	b
1100	12	c
1101	13	d
1110	14	e
1111	15	f

<그림23_SW문제해결 응용_시작하기_진법 변환4>

2진수 1100101111를 16진수로 변환해보기

- 먼저 네 자리씩 끊어주기
  - 11 0010 1111
- 정답 : 0x32F

2진수	10진수	16진수
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	8
1001	9	9
1010	10	a
1011	11	b
1100	12	c
1101	13	d
1110	14	e
1111	15	f

<그림23_SW문제해결 응용_시작하기_진법 변환4>

16진수를 입력받으면, 2진수로 출력해주는 프로그램

- 오른쪽 표에서 2진수 부분을 리스트에 모두 저장해두고 구현

2진수	10진수	16진수
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	8
1001	9	9
1010	10	a
1011	11	b
1100	12	c
1101	13	d
1110	14	e
1111	15	f

<그림23_SW문제해결 응용_시작하기_진법 변환4>

0과 1로 이루어진 1차 배열에서 7개씩 수를 묶어, 10진수로 출력하기

- 예시

0	0	0	0	0	0	1	0	0	0	1	1	0	1

<그림24_SW문제해결 응용_시작하기_진법 변환5>

이면 1, 13 을 출력한다.

- 입력 예
  - ✓ 입력 예시 : 00000001111000000110000001111001100001100000111100111100111111001100111
  - ✓ 출력 결과 : 0 120 12 7 76 24 60 121 124 103

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

비트 연산

비트와 바이트

- 1 bit : 0과 1을 표현하는 정보의 단위
- 1 Byte : 8-bit를 묶어 1 Byte라고 함 (메모리 주소가 부여되는 단위)

- 예시

  1001011011011100 은 총 몇 비트이면서, 몇 바이트 인가?

비트 연산

- 컴퓨터의 CPU 내부적으로 비트 연산을 사용하여 덧셈, 뺄셈, 곱셈 등을 계산함

- 비트 연산 챕터의 목적

  - 사람이 사용하는 사칙연산 (+, *, /, -)이 아닌 컴퓨터가 사용하는 연산인 “비트 연산”을 이해해봅니다.
  - 더 나아가, 프로그래밍에서 비트연산을 활용한 코딩 방법을 익혀봅니다.

비트 연산 AND와 OR 이해하기

- a AND b : a, b 둘 다 1 일 때만 결과가 1, 그 외에는 0
- a OR b : a, b 둘 중 하나만 1 이면 결과가 1, 그 외에는 0

연산자	연산자의 기능
&	비트 단위로 AND 연산을 한다. 예) num1 & num2
|	비트 단위로 OR 연산을 한다. 예) num1 | num2

<표2_SW문제해결 응용_시작하기_연산자의 기능>

파이썬 구현

- 7 & 5 -> 이진수로 표현하면 0b111 & 0b101

```
    111
&   101
  ----
    101
```

&(AND)
둘 다 1일 때 1

```
    111
|   101
  ----
    111
```

| (OR)
하나만 1이면 1

```
print(7 & 5)
print(7 | 5)
```

```
5
7
```

<그림25_SW문제해결 응용_시작하기_비트연산 파이썬 구현>

파이썬에서 2진수, 16진수, 10진수 변환하여 출력하기

- 2진수는 숫자 0과 소문자 b -> 0b를 접두사로 붙여 표현합니다.
- 16진수는 숫자 0과 소문자 x -> 0x를 접두사로 붙여 표현합니다.

```
print(bin(10))
print(hex(10))
```

```
0b1010
0xa
```

10진수를 2, 16진수로 변환

```
print(int('1011', 2)) #2진수
print(int('b', 16)) #16진수
```

```
11
11
```

2, 16진수를 10진수로 변환

<그림26_SW문제해결 응용_시작하기_비트연산 파이썬 구현2>

[도전] 먼저 수기로 연산을 해본다.

- 이후, 직접 파이썬으로 구현하여 테스트

- 0b11011110 & 0b11011

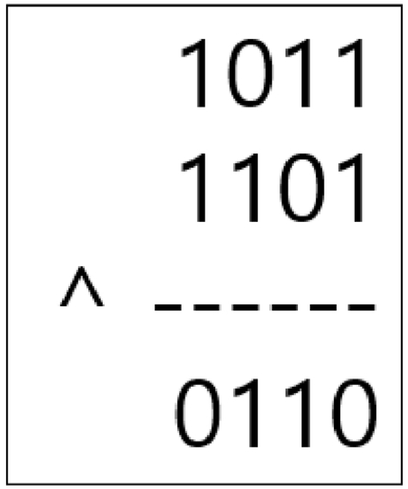
- 0x4A3 | $25_{(10)}$

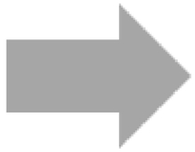
비트 XOR 연산자

- ^ : XOR(Exclusive-OR) 연산자, 둘 다 1이거나 0인 경우는 0이다.

연산자	연산자의 기능
^	비트단위로 XOR 연산을 한다. (같으면 0 다르면 1) 예) num1 ^ num2

<표3_SW문제해결 응용_시작하기_연산자의 기능2>





```
a = bin(0b1011 ^ 0b1101)

print(a)
```

```
0b110
```

<그림27_SW문제해결 응용_시작하기_XOR1>

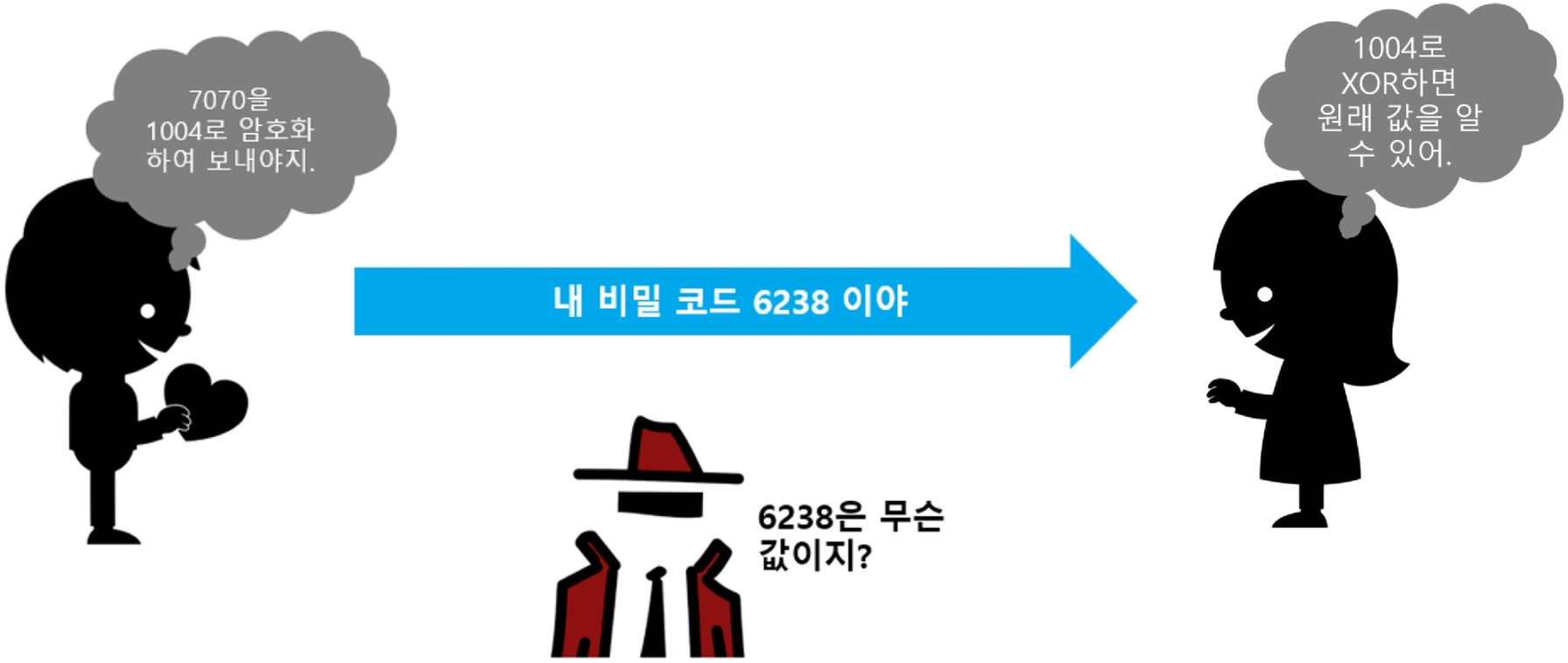
신기한 XOR

- 어떤 값이던 특정 수로 2회 XOR를 하면 원래 수로 돌아옴

- 7070 ^ 1004 = 6258
- 6258 ^ 1004 = 7070

암호화에 사용되는 XOR

- 친구1과 친구2는 서로 비밀코드를 1004로 약속



<그림28_SW문제해결 응용_시작하기_XOR2>

[도전] 암호화 프로그램 제작하기

- 수를 입력받고, 암호화(인코딩)를 하거나 복호화(디코딩)하는 프로그램을 제작
- KEY 값은 1004로 정함

- 예시 1)

  E 1000 # 수 1000을 인코딩하여 출력한다.

- 예시 2)

  D 4 # 수 4를 디코딩하여 출력한다.

쉬프트 연산자

- Left Shift << : 특정 수 만큼 비트를 왼쪽으로 밀어냄
- Right Shift >> : 특정 수 만큼 비트를 오른쪽으로 밀어냄 (우측 비트들이 제거됨)

연산자	연산자의 기능
<<	피연산자의 비트 열을 왼쪽으로 이동시킨다. 예) num << 2 #2회 이동
>>	피연산자의 비트 열을 오른쪽으로 이동시킨다. 예) num >> 2 #2회 이동

<표4_SW문제해결 응용_시작하기_연산자의 기능3>

```
print(bin(0b1101 << 2))
print(bin(0b1101 >> 2))
```

```
0b110100
0b11
```

<그림29_SW문제해결 응용_시작하기_비트 연산자>

[도전] Left Shift (<<) 를 이용한 프로그래밍

- 반복문을 이용하여 아래와 같이 출력
- Loop 1 : 0b1 출력 (2진수와 10진수로 출력)
- Loop 2 : 0b10 출력 (2진수와 10진수로 출력)
- Loop 3 : 0b100 출력 (2진수와 10진수로 출력)
- Loop 4 : 0b1000 출력 (2진수와 10진수로 출력)
- Loop 5 : 0b10000 출력 (2진수와 10진수로 출력)

```
0b1 1
0b10 2
0b100 4
0b1000 8
0b10000 16
```

출력 결과

<그림30_SW문제해결 응용_시작하기_Left Shift>

비트 연산 응용 1

- 1 << n

  - 2^n 의 값을 갖습니다.

  - 임베디드 분야에서 계산을 빠르게 하기 위해 사용됩니다.

비트 연산 응용 2

- i & (1 << n)

  - i의 n번째 비트가 1인지 아닌지를 확인할 수 있습니다.

  - ex) 1101 & (1 << 2)
    1101에서 2번 bit가 1인지 확인 가능합니다. (결과값 = 0100)
    결과값이 0보다 크면 n 번째 비트는 1임이 확정됩니다.

음수 표현 방법

- 컴퓨터는 음수를 “2의 보수”로 관리함
  맨 앞자리 bit(MSB)는 음수 or 양수를 구분하는 비트임
- 컴퓨터가 2의 보수를 사용하여 음수를 관리하는 이유

- 뺄셈의 연산 속도를 올릴 수 있으며, +0과 -0을 따로 취급하지 않기 위해 사용합니다.

```
def bit_print(i):
    output = ‘’
    for j in range(7, -1, -1):
        output += '1' if i & (1 << j) else '0’
        print (output)

for i in range(-5, 6):
    print (f"{i} = ", end=‘’)
    bit_print(i)
```

```
-5 = 11111011
-4 = 11111100
-3 = 11111101
-2 = 11111110
-1 = 11111111
0 = 00000000
1 = 00000001
2 = 00000010
3 = 00000011
4 = 00000100
5 = 00000101
```

<그림31_SW문제해결 응용_시작하기_음수 표현 방법>

2의 보수 예시

- 10001의 2의 보수

-> 수를 모두 뒤집고 1을 더합니다.
-> 01110 + 1 = 01111

- 1111000의 2의 보수

-> 수를 뒤집으면 0000111이고 1을 더합니다.
-> 0000111 + 1 = 0001000

신기한 2의 보수

- 2의 보수를 취한 수를, 한 번 더 2의 보수를 취하면 원래의 값으로 돌아옴

- 10001의 2의 보수
  -> 수를 모두 뒤집고 1을 더합니다.
  -> 01110 + 1 = 01111

- 01111의 2의 보수

  -> 10000 + 1 = 10001

-5를 2의 보수로 표현하는 방법 (가정 : 수를 8-bit로 저장하는 경우)

- 수 5를 2진수로 나타내면 0000 0101 (7-bit)
- -5는 음수이기에 MSB는 1
- 나머지 7-bit에 대해, 수를 뒤집고 1을 더함 (2의 보수)
- 수 5를 뒤집으면 111 1010 이며, 1을 더하면 111 1011이 됨
- 따라서 1111 1011 이 됨

```
-5 = 11111011
-4 = 11111100
-3 = 11111101
-2 = 11111110
-1 = 11111111
0 = 00000000
1 = 00000001
2 = 00000010
3 = 00000011
4 = 00000100
5 = 00000101
```

<그림31_SW문제해결 응용_시작하기_음수 표현 방법>

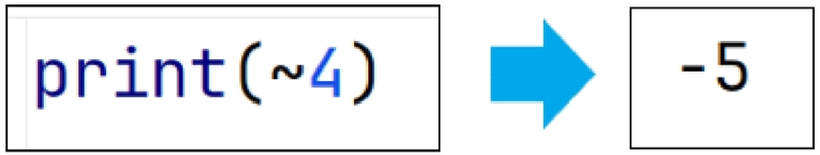
Bitwise NOT(complement) 연산자

- ~ 연산자 : 모든 비트를 반전시킴
- 만약 8-bit 일 때 ~(0001 1111) 이라면 값은 1110 0000 이 됨

Bitwise NOT 연산자를 파이썬에서 수행하기

- 파이썬에서는 ~4 를 수행하면 -5가 출력됨

- **파이썬이 ~4를 -5로 출력하는 과정**
  - 4는 0b0100입니다. (MSB : 양수이므로 0)
  - NOT 연산자로 인해 뒤집으면 1011이 됩니다.
  - MSB는 1이 되었고 (음수), 나머지 bit는 011입니다.
  - 나머지 bit에 대해 2의 보수를 취하면 100 + 1 = 101이므로 5가 됩니다.
  - 따라서 -5가 됩니다.



<그림32_SW문제해결 응용_시작하기_Bitwise NOT>

[도전] 비트연산 문제 풀어보기 (SWEA 10726. 이진수 표현)

- 정수 N, M 이 주어질 때, M의 이진수 표현의 마지막 N 비트가 모두 1로 켜져 있는지 아닌지를 판별하여 출력, 모두 켜 있다면 ON 출력, 아니면 OFF 출력

- ex)
  5 31
  -> 31은 이진수로 0001 1111 입니다.
  -> 5개 bit가 모두 1로 켜 있습니다.
  -> 따라서 정답은 “ON”

IP 주소가 192.168.1.130, netmask가 255.255.255.192(/26)인 PC가 속한 서브넷의 범위는?

- Netmask가 255.255.255까지는 모든 비트가 1로 192.168.1은 고정
- 192는 2진수로 표현하면 1100 0000
- IP 중 130을 2진수로 표현하면 1000 0010
- 마스크에서 1로 표시된 비트는 IP가 고정이므로, 0인 6개의 비트만 고려하면 1000 0000(128)부터 1011 1111(191)
- 따라서 서브넷의 범위는 192.168.1.128 ~ 192.168.1.191이 된다.

실수

파이썬에서 실수 출력 방법

- 파이썬은 f-string 문법을 지향

```
t1 = 10
t2 = 3.141592

print(f'변수 값은 {t1} 입니다')
print(f'변수 값은 {t2} 입니다')
```

```
변수 값은 10 입니다
변수 값은 3.141592 입니다
```

<그림33_SW문제해결 응용_시작하기_파이썬에서 실수 출력>

소수점 출력 방법

- {t2:.2f} : t2 값을 소수점 셋째자리에서 반올림하여 표현

```
t1 = 10
t2 = 3.141592

print(f'변수 값은 {t1} 입니다')
print(f'변수 값은 {t2:.2f} 입니다')
```

```
변수 값은 10 입니다
변수 값은 3.14 입니다
```

<그림34_SW문제해결 응용_시작하기_파이썬에서 실수 출력2>

파이썬에서의 실수 표현 범위

- 파이썬에서는 64비트 부동소수점으로 실수를 표현함

  - 15~17자리 정밀도와 ±1.8e308 범위를 갖습니다.

- 최대로 표현할 수 있는 값은 약 1.8×10^{308} 이고 이 이상은 inf로 표현
  (참고로 1억은 1×10^{8} 임)
- 최소로 표현할 수 있는(0에 가장 가까운) 값은 약 5.0×10^{-324} 이며, 이 이하는 0으로 표현

컴퓨터는 실수를 내부적으로 근사적으로 관리

- 실수는 정확한 값이 아니라 근사 값으로 저장되는데 이때 생기는 작은 오차가 계산 과정에서 다른 결과를 가져옴

```
print(0.1+0.1+0.1 == 0.3)
```

```
False
```

<그림35_SW문제해결 응용_시작하기_실수1>

[도전] 실수 값 출력해보기

- 0.1이 정확히 어떤 값으로 저장되었는지, f-string 을 이용하여 출력해보자.
- 소수점 20자리 이상으로도 출력해보자.

```
t = 0.1
print(f'{t:.2f}')
```

```
0.10
```

<그림36_SW문제해결 응용_시작하기_실수2>

소수점이 있는 10진수를 2진수로 변환 예시

- $0.75 = 2^{-1} + 2^{-2}$

  $= 0.11_{(2)}$

- $0.40625 = 2^{-2} + 2^{-3} + 2^{-5}$

  $= 0.01101_{(2)}$

2^{-5}	$= 0.03125$
2^{-4}	$= 0.0625$
2^{-3}	$= 0.125$
2^{-2}	$= 0.25$
2^{-1}	$= 0.5$
2^{0}	$= 1$

<그림37_SW문제해결 응용_시작하기_실수3>

소수점을 포함한 2진 실수를 10진수로 변환하는 예시

- 예) 1001.0011

1 x 2^{-4}	= 1 x 0.0625	= 0.0625
1 x 2^{-3}	= 1 x 0.125	= 0.125
0 x 2^{-2}	= 0 x 0.25	= 0
0 x 2^{-1}	= 0 x 0.5	= 0
1 x 2^{0}	= 1 x 1	= 1
0 x 2^{1}	= 0 x 2	= 0
0 x 2^{2}	= 0 x 4	= 0
1 x 2^{3}	= 1 x 8	= 8

총합이 10진수로 9.1875

실수의 표현

- 컴퓨터는 실수를 표현하기 위해 부동 소수점(floating-point) 표기법을 사용
- 이 표기법은 IEEE 754라는 국제표준(IEEE 754-2019, 2029년 개정 예정)

- 부동 소수점 표기 방법은 소수점의 위치를 고정시켜 표현하는 방식
- 소수점의 위치를 왼쪽의 가장 유효한 숫자 다음으로 고정시키고 밑 수의 지수 승으로 표현

$$1001.0011 \rightarrow 1.0010011 \times 2^3$$

실수를 저장하기 위한 형식 (IEEE 754, 32bits - Single Precision 표기법 기준)

- 32-bit 구조

부호1비트	지수 8비트	가수 23비트

<그림38_SW문제해결 응용_시작하기_32bit 구조>

- 부호 1비트 : 0 이면 양수, 1 이면 음수
- 지수부(exponent) : 부동소수점의 크기 + bias값
- 가수부(mantissa) : 실질적 수

이론

12.375를 부동소수점으로 표기

- 12.375를 2진수로 변환 : 12는 1100, 0.375는 0.011이므로 1100.011
- 1100.011 = 1.100011×2^3
  즉, 가수는 100011이고, 지수는 3. (맨 앞 1은 생략)

지수 + bias : IEEE 754는 bias를 더한 결과로 표기

- 3 + 127(bias) = 130 (2진수로는 10000010)

12.375를 IEEE 754 로 표기하는 방법

- 부호 비트: 0 (양수)
- 지수: 10000010
- 가수: 10001100000000000000000

이론

컴퓨터는 실수를 근사적으로 표현

- 이진법으로 표현할 수 없는 형태의 실수는 정확한 값이 아니라 근사 값으로 저장되는데 이때 생기는 작은 오차가 계산 과정에서 다른 결과를 가져옴

실수 자료형의 유효 자릿수

- 32비트 실수형 유효자릿수(10진수) -> 약 6자리 (C/C++, Java)
- 64비트 실수형 유효자릿수(10진수) -> 약 15자리 (C/C++, Java, Python)

연습 문제

16진수 문자로 이루어진 1차 배열이 주어질 때 앞에서부터 7-bit씩 묶어 10진수로 변환하여 출력하기

- 예를 들어

0	F	9	7	A	3

<그림39_SW문제해결 응용_시작하기_연습 문제1>

일 경우, 2진수로 변환하면

000011111001011110100011 이고, 이를 7-bit씩 묶으면

000011111001011110100011 이므로, 출력 결과는 7, 101, 116, 3이다.

- 예시

입력 : 01D06079861D79F99F

정답 : 14 65 64 121 67 7 47 31 76 31

16진수 문자로 이루어진 1차 배열이 주어질 때, 왼쪽부터 순차적으로 암호 비트패턴을 찾아 차례대로 출력하시오. 암호는 연속되어 있다.

- 예를 들어

0	D	E	C

<그림40_SW문제해결 응용_시작하기_연습 문제2>

일 경우, 2진수로 변환하면

0000110111101100 이고,

암호 패턴을 왼쪽부터 순차적으로 찾으면
00001101111011**00** 이 되므로,
정답은 0 2 이다.

- 입력 예
  - 0269FAC9A0

암호비트패턴

0	001101
1	010011
2	111011
3	110001
4	100011
5	110111
6	001011
7	111101
8	011001
9	101111

<표5_SW문제해결 응용_시작하기_암호 비트패턴>

실습

진법

- 5185. 이진수
- 5186. 이진수2

추가 연습

- 1240. 단순 2진 암호코드
- 1242. 암호코드 스캔

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

1 SW 문제해결 역량이란?

a) 주어진 문제를 수학적으로 해결하는 능력
b) 소프트웨어로 문제를 해결하는 전반적인 사고와 기술
c) 프로그래밍 언어를 빠르게 배우는 능력
d) 컴퓨터를 조립하고 다루는 능력

2 아래 중 문제해결 과정에 해당하지 않는 것은?

a) 문제 이해
b) 계획 세우기
c) 무작위 테스트
d) 해결 방법 구현

3 아래 코드에서 출력되는 결과는?

```
x = 7
print(x << 2)
```

4 아래 코드에서 출력되는 결과는?

```
a = 12
b = 5
print(a & b)
```

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

5 아래 코드에서 출력되는 결과는?

```
num = 10
print(~num)
```

6 다음 중 시간 복잡도가 가장 높은 것은?

a) O(n)
b) O(n log n)
c) O(n²)
d) O(log n)

7 평균적으로 O(n log n) 알고리즘의 예로 적절한 것은?

a) 버블 정렬
b) 퀵 정렬
c) 선형 탐색
d) 이진 탐색

8 아래 코드에서 출력되는 결과는?

```
print(0o17 + 0xA)
```

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

9 파이썬에서 int("0x1A", 16)의 결과는?

a) 26
b) 1A
c) 16
d) 오류 발생

10 아래 코드에서 출력되는 결과는?

```
x = 0b1001
print(x >> 2)
```

11 아래 코드에서 출력되는 결과는?

```
def bit_count(n):
    return bin(n).count('1')
print(bit_count(15))
```

12 2진수 1010을 10진수로 변환하면?

a) 5
b) 10
c) 12
d) 8

확인문제

정답 및 해설

1. b) 소프트웨어로 문제를 해결하는 전반적인 사고와 기술 2. c) 무작위 테스트 3. 28 4. 4 5. -11 6. c) $O(n^2)$ 7. b) 퀵 정렬 8. 25 9. a) 26 10. 2 11. 4 12. b) 10

1. SW문제해결 능력이란 소프트웨어로 문제를 해결하는 전반적인 사고와 기술을 말합니다.

2. 문제해결 과정은 아래와 같습니다.
   1. 문제를 읽고 이해한다.
   2. 문제를 익숙한 용어로 재정의한다.
   3. 어떻게 해결할지 계획을 세운다.
   4. 계획을 검증한다.
   5. 프로그램으로 구현한다.
   6. 어떻게 풀었는지 돌아보고, 개선할 방법이 있는지 찾아본다.

3. 2칸 왼쪽을 시프트 하여 $7 * 2^2 = 28$ 입니다.

4. AND 연산 결과: 1100 & 0101 -> 0100 이므로 답은 4입니다.

5. ~은 비트 NOT 연산자입니다. 즉, ~X = -X -1 = -11 입니다.

6. 보기 중 시간 복잡도가 가장 높은 것은 $O(n^2)$ 입니다.

7. 퀵 정렬은 O(n log n) 알고리즘의 예입니다.

8. 8진수 0o17 = 15, 16진수 0xA = 10 이므로 출력 결과인 합은 25입니다.

9. int("0x1A", 16)의 결과는 26입니다.

10. 2칸 오른쪽 시프트 하여 1001 -> 0010 으로, 출력 값은 2가 됩니다.

11. 15 = 1111으로, 4개의 1이 카운트 되어 4입니다.

12. 2진수 1010을 10진수로 변환하면 10입니다.

활동정리

핵심키워드

개념	설명	예시
빅오 표기법	복잡도의 점근적 상한	O(1), O(n), O(log n)
비트 연산	데이터에 대한 비트 단위 연산	&, |, ^, ~, <<, >>
진법 변환	숫자를 다른 진법으로 바꾸는 계산	10진수 ↔ 2진수, 16진수
실수	Python은 64비트 부동소수점으로 표현	3.14
XOR 암호화	비트 XOR 연산을 두 번 반복하면 원래 값으로 복귀하는 성질 이용	a ^ key → 암호화

<표6_SW문제해결 응용_시작하기_핵심키워드>

활동정리

오늘 공부한 내용 요약 및 정리

- **알고리즘과 복잡도**
  - 알고리즘은 문제를 해결하는 명확한 절차이며, 복잡도는 이 알고리즘이 얼마나 효율적인지를 나타내는 지표이다.

- **파일 입력 처리**
  - 반복되는 입력을 자동화하고, 대규모 데이터 처리에 효과적인 방법이다.

- **진법 변환**
  - 컴퓨터 분야에서 자주 사용되는 2진법과 16진법을 이해한다.

- **비트 연산**
  - 다양한 비트연산을 사용해 연산시간을 단축시키거나 암호화에 사용할 수 있다.

활동정리

‘컴퓨터 입장에서 본 SW 문제해결 시작하기’

1. 문제를 해결할 때 항상 [ ] 절차를 따릅니다.
2. 사람들은 나를 빠르게 만들기 위해 [ ] 를 분석해서가능한 적은 [ ] 안에 계산이 끝나도록 알고리즘을 설계해줍니다.
3. 어떤 입력이 주어져도 나는 [ ] 에 따라 정확히 실행만 하면 됩니다.

활동정리

‘컴퓨터 입장에서 본 SW 문제해결 시작하기’

1. 문제를 해결할 때 항상 정해진 절차를 따릅니다.
2. 사람들은 나를 빠르게 만들기 위해 복잡도를 분석해서가능한 적은 시간 안에 계산이 끝나도록 알고리즘을 설계해줍니다.
3. 어떤 입력이 주어져도 나는 알고리즘에 따라 정확히 실행만 하면 됩니다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

감사합니다